Chapter -2

Operating System Structures

- This chapter deals with how operating systems are structured and organized.
- Different design issues and choices are examined and compared, and the basic structure of several popular OSes are presented.

Operating System Components

An **Operating System (OS)** provides an environment where programs can run.

- It is very large and complex, it is divided into smaller components, each responsible for a specific job.
- It makes sure all programs run smoothly and can use the computer's resources safely.
- Before building an OS, we must know its purpose and goals. The type of OS affects how it is designed.

When we want to build an operating system, we first think about what kind of OS we want

For example:

- **Do we want it to be fast?**
- **Able to handle many users?**
- **For mobile or computer?**

After deciding the goals, we break the OS into small modules like memory manager, process manager, file manager, etc.

Each module has a specific job and a clear interface to talk with other modules.

Not all systems have the same structure.

Many modern **OS share the system components** outlined below.

- Process management
- I/O management
- Main Memory management
- File & Storage Management
- Protection
- Networking
- Command Interpreter

Process management

- A process is a program in execution.
- It can be suspended temporarily and the execution of another process can be taken up.
- Create, Load, execute, suspend, resume and terminate processes
- Switch system among multiple processes in main memory (Process Scheduling)

I/O management

- Hide the details of hardware devices from the application programmer.
- Manage main memory for the device using cache, buffer.
- Maintain and manage device driver interface

Main Memory Management

- Maximize memory Utilization
- Keep Track of which memory area is used by whom.
- Allocate/deallocate memory as needed

File & Storage Management

- Create, manipulate, delete files and directories
- Allocate, de-allocate, and defrag blocks

Protection

- The security mechanisms in an operating system ensure that authorized programs have access to resources, and unauthorized programs have no access to restricted resources.
- Security management refers to the various processes where the user changes the file, memory, CPU, and other hardware resources that should have authorization from the operating system.

Networking

- It is a software application that provides network administrators with information on components in their networks.
- It ensures the quality of service and availability of network resources.
- It also examines the operations of a network, reconstructs its network configuration, modifies it for improving performance of tasks.

Command Interpreter

- Several ways for users to interface with the OS. One of the approaches to user interaction with the operating system is through **CLI.**
- Allow users to interact with OS.
- Provide convenient programming environment to users.
- Execute a user command by calling one or more number.

Operating System Services

An OS provides an environment for the execution of programs. It provides certain services to programs and to the users of those programs.

Following are the **functions provided by OS**:

1. **User interface:** almost all OS have set of user interface (UI). It can be of three types:

    a. Command-line interface (CLI): which uses **text commands** and a method for entering them.

    b. Graphical user interface: Interface is a window system with a **pointing device to direct I/O**, choose from menu, and make selections and a keyboard to enter text. (commonly use, user friendly)

    c. Touch-screen interface: enabling users to slide their fingers across.

        the screen or press buttons on the screen to select choices.

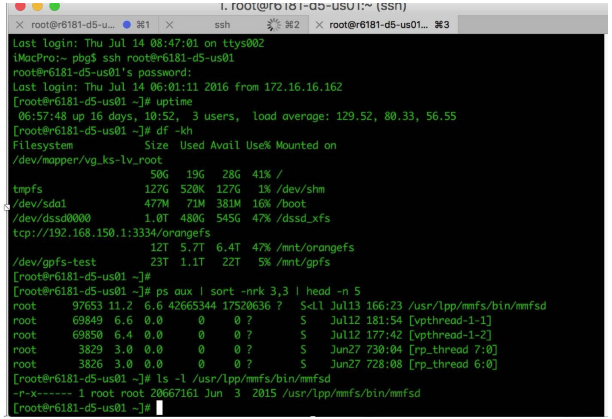
1) User Interface

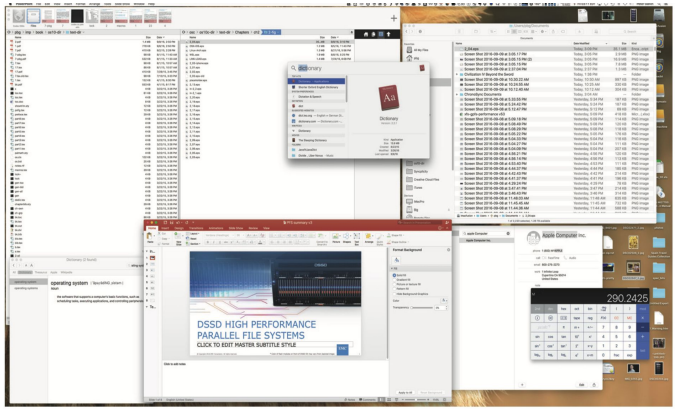
Command Line Interface (CLI)

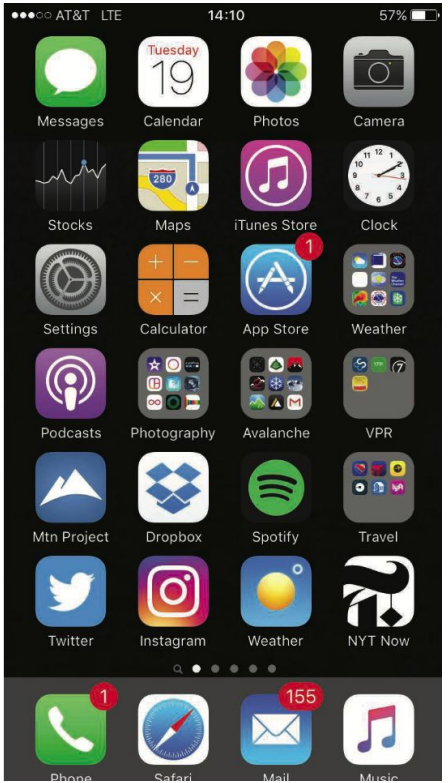
Graphical User Interface (GUI)

User and Operating-System Interface

- Command Interpreters
- Graphical User Interface
- Touch-Screen Interface







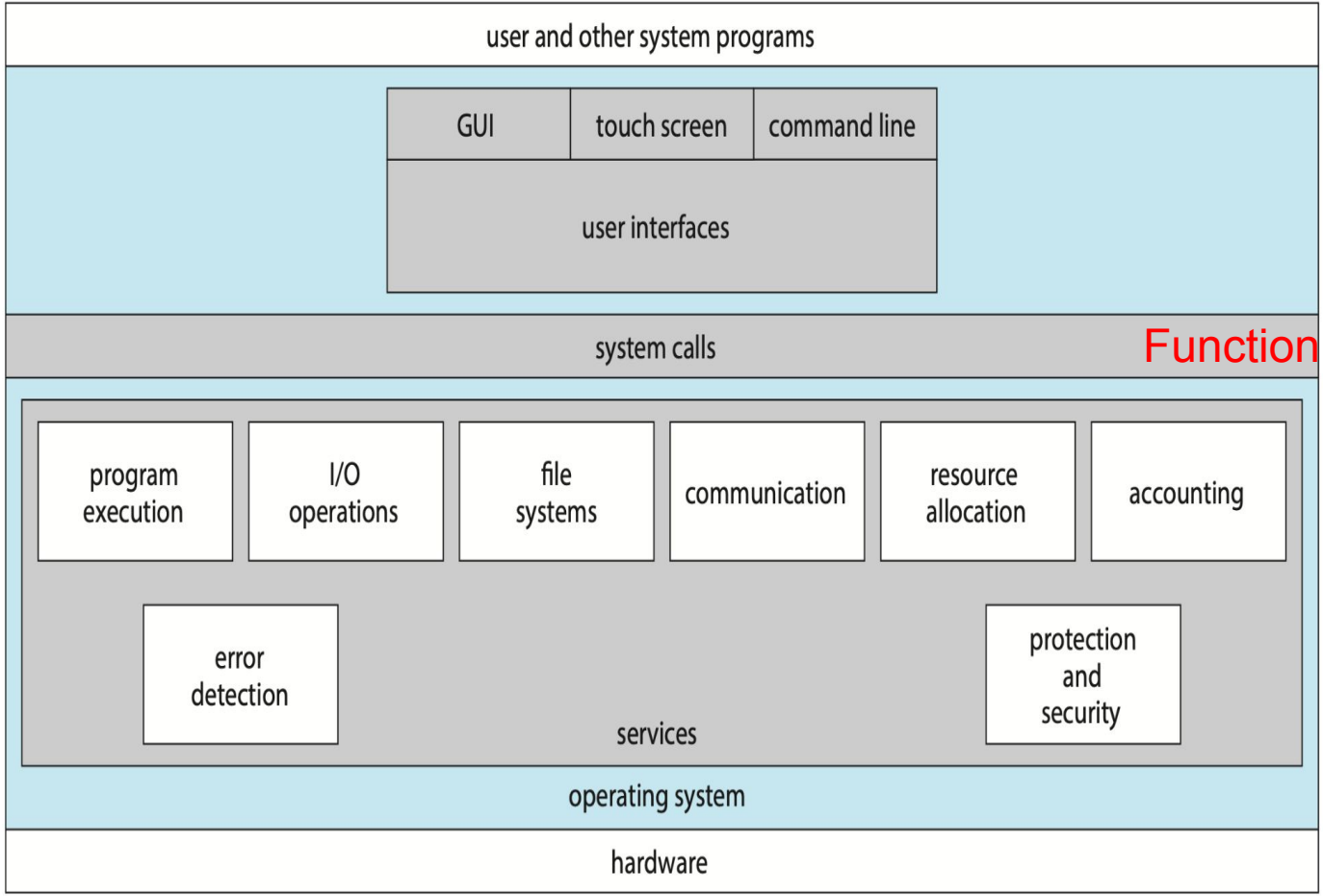


Fig: A view of operating system

applications and users interact with the OS

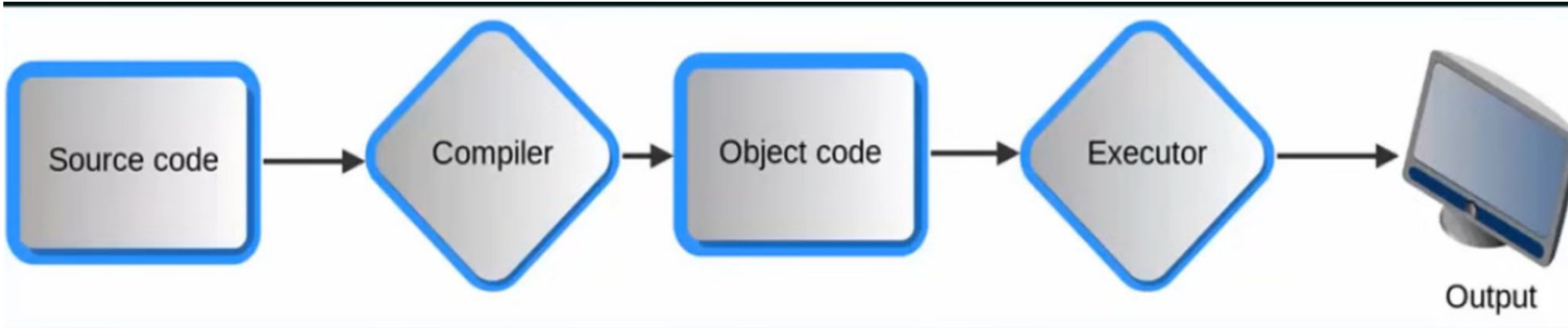
User Interface

user program or application asks the operating system (OS) to do something for it.

Services Layer

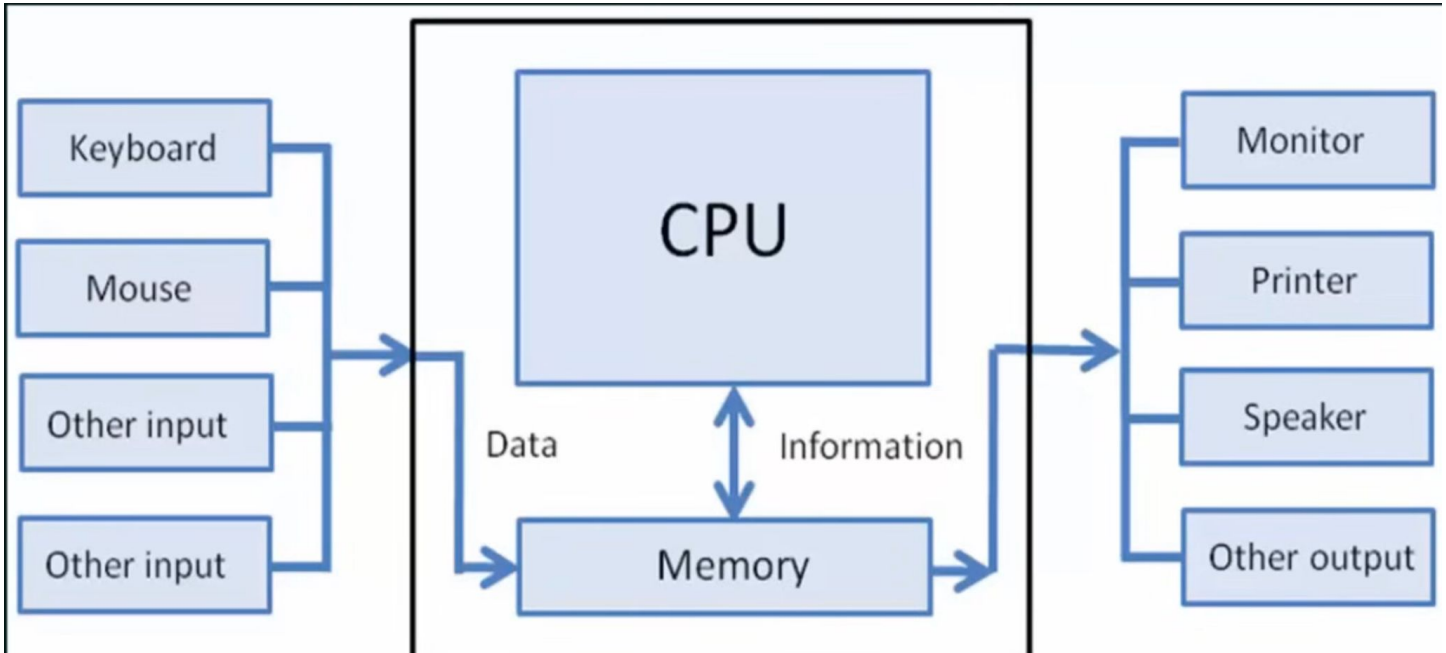
h/w -> physical part : CPU, memory, hard drives, keyboard, mouse, etc. needs os to manage

Program Execution - The OS must be able to **load a program into RAM**, run the program, and terminate the program, either normally or abnormally.



I/O Operations - The OS is responsible for **transferring data to and from I/O devices**, including keyboards, terminals, printers, and storage devices.

User cannot directly control the I/O device : OS is btw user and system



File-System Manipulation - Need to **create and delete** them by name, search for a given file, and list file information, permission management to allow or deny access to files or directories based on file ownership.

In addition to raw data storage, the OS is also responsible for **maintaining directory and subdirectory structures,** mapping file names to specific blocks of data storage, and providing tools for navigating and utilizing the file system.

Communications - Inter-**process** communications, either between processes running on the same computer, or between processes running on separate computer tied by Computer Network. May be implemented as either shared memory or message passing, (or some systems may offer both.)

Error Detection - Both hardware and software errors must be detected and handled appropriately, with a minimum of harmful repercussions.

Error on CPU, h/w, memory, I/O device. *Eg: printing and out of paper.*

The OS must have a way in which it manages those error. So that computing are consistent .

Resource Allocation - Allocating Resources to different Processes at particular time.

Resources = CPU, Main Memory, Files

Many process is running in system -> all process required certain resource at certain Point of time.

OS must allocate the required resources to the processes which are waiting or asking for those resources. It must allocate them -> efficient -> so that no process keep waiting and never get it. (shouldn't have such scenario)

Accounting - Keep track of which user use **how much** and **what kind** of **Computer Resources.**

Protection & Security - Protection: Means that access to the system resources must be controlled.

Security: The system is not accessible to outsiders.

User Mode and Kernel mode

User mode is a restricted, **low-privilege, slave mode or restricted** state where user applications run,

while **kernel** mode is a **privileged, Supervisory** state with full access to system hardware and resources. [i.e: bcz it having direct access to many resources (memory, h/w-> if program crash -> entire system on halt]

- **User mode does not have access** to h/w components (safer mode)
- **Kernel mode has direct access** to hardware components.
- When process execute in user mode it may requires h/w resources,

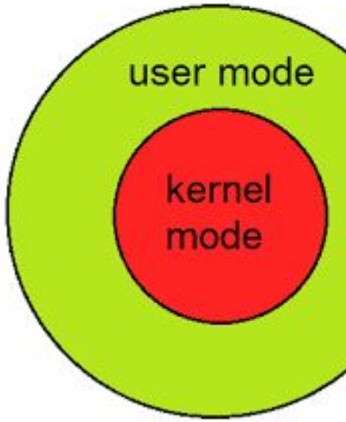
  Like Memory, H/w ; when program need access to certain resource

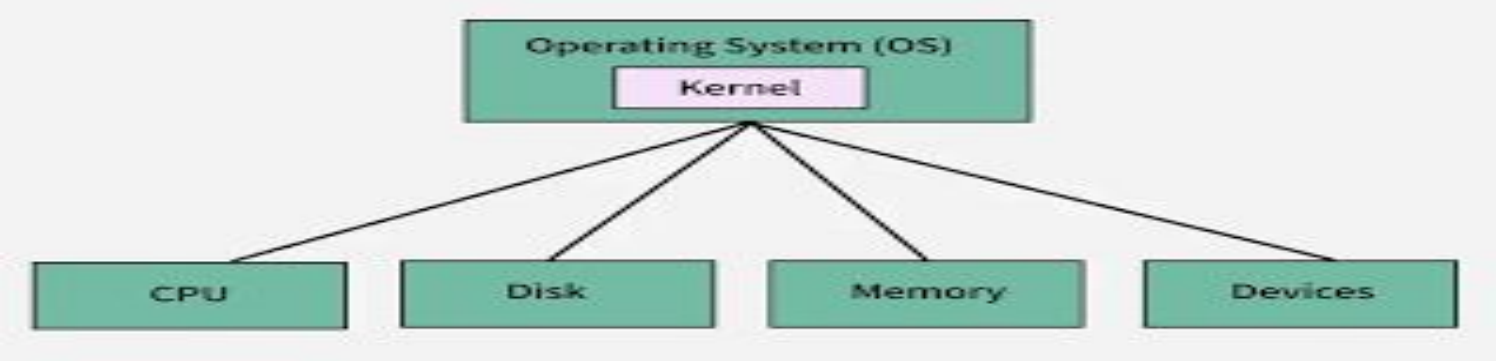
  send a request to kernel & programis switched from UM to KM(Context switching

- Context switching : Switching from user mode to kernel mode.

System call are made by the program when it needs to access certain resources.

It is the programmatic way in which a computer program requests a service from the kernel of OS.





Kernel – The Kernel is the core part of the operating system. It manages communication between the hardware and the software

Manages CPU, memory, devices, and processes

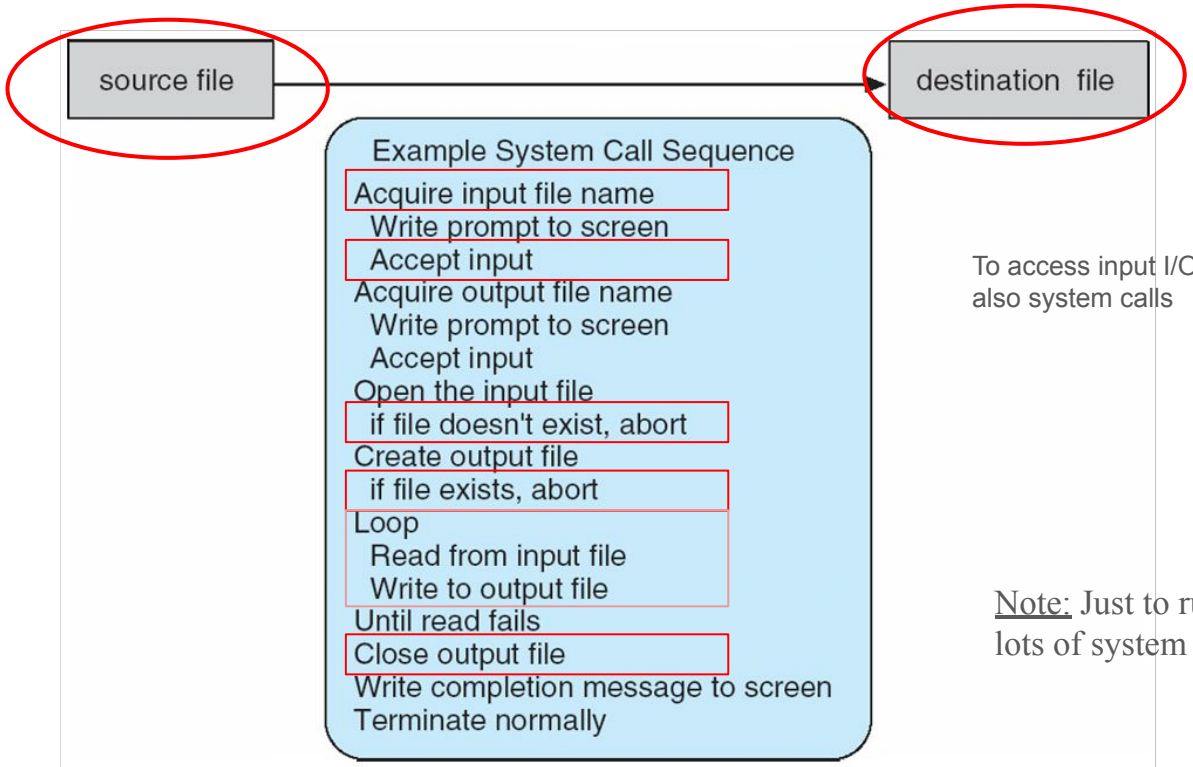
Can be monolithic, microkernel, or hybrid

User Interaction	Users directly interact with the OS through interfaces like GUIs or command lines.	The kernel operates in the background, with no direct user interaction.

User Mode vs Kernel Mode	
User Mode is a restricted mode, which the application programs are executing and starts out.	Kernel Mode is the privileged mode, which the computer enters when accessing hardware resources.
Modes	
User Mode is considered as the slave mode or the restricted mode.	Kernel mode is the system mode, master mode or the privileged mode.
Address Space	
In User mode, a process gets their own address space.	In Kernel Mode, processes get single address space.
Interruptions	
In User Mode, if an interrupt occurs, only one process fails.	In Kernel Mode, if an interrupt occurs, the whole operating system might fail.
Restrictions	
In user mode, there are restrictions to access kernel programs. Cannot access them directly.	In kernel mode, both user programs and kernel programs can be accessed.

Example of System Calls

- System call sequence to copy the contents of one file to another file



To access input I/O device keyboard mouse for that also system calls

Note: Just to run simple task of copying run lots of system calls involved.

System Call: Copy the Content of One File to Another

In an operating system, system calls are used to request services from the kernel (OS).
To copy a file, we use three important system calls:

1. open() – to open the source and destination files

2. read() – to read data from the source file

3. write() – to write that data into the destination file

4. close() – to close both files

Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory

- System call used for the controlling the processes control.

Eg: when a process is running it has to terminate halt either **normally** or **abnormally**

Process Complete -> Terminate Normally

Error occur in process execution -> it has to be halted or aborted -> that is called abnormally

Types of System Calls (cont.)

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Managing or manipulating device

Eg: When you click **Print** on your computer, behind the scenes the OS uses

Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Information about system must be update and maintain.

Communication between processes

Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

One user cannot access another user's files
One process cannot interfere with another process
Unauthorized programs cannot access hardware, memory, or data

Some eg:

chmod()	Change file permissions
chown()	Change file owner

Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

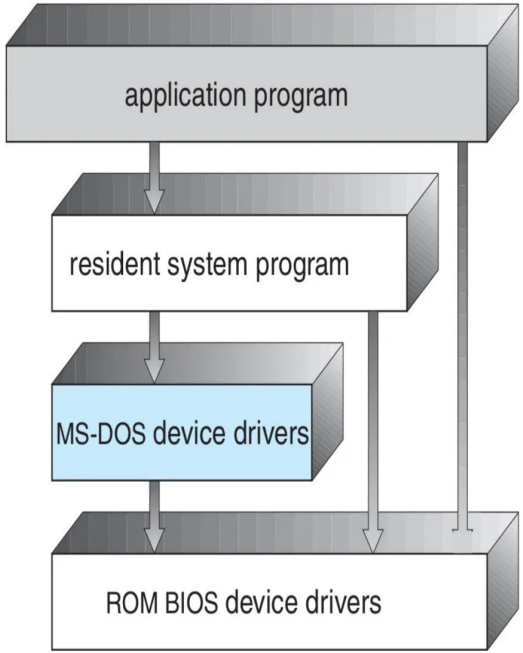
The following illustrates various equivalent system calls for Windows and UNIX operating systems.

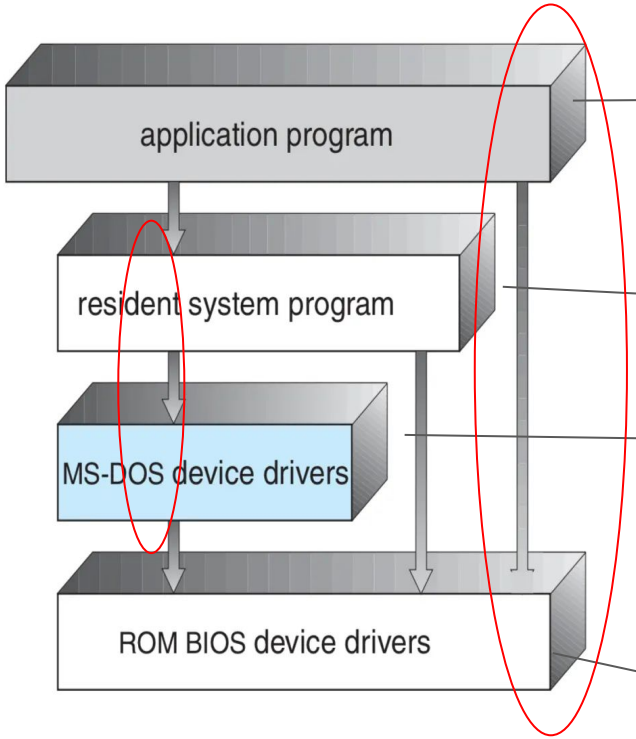
	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

Simple Structure

Many operating systems do not have well-defined structures.

- Used to design **initial Operating System.** Eg: MS- DOS, UNIX
- More Functionality in less space.
- Structure was not well defined but also got popular.





Program where user directly interact *eg:* write document - Ms Word

E.g: file manipulation

Program that operate or controls H/W device attached to computer

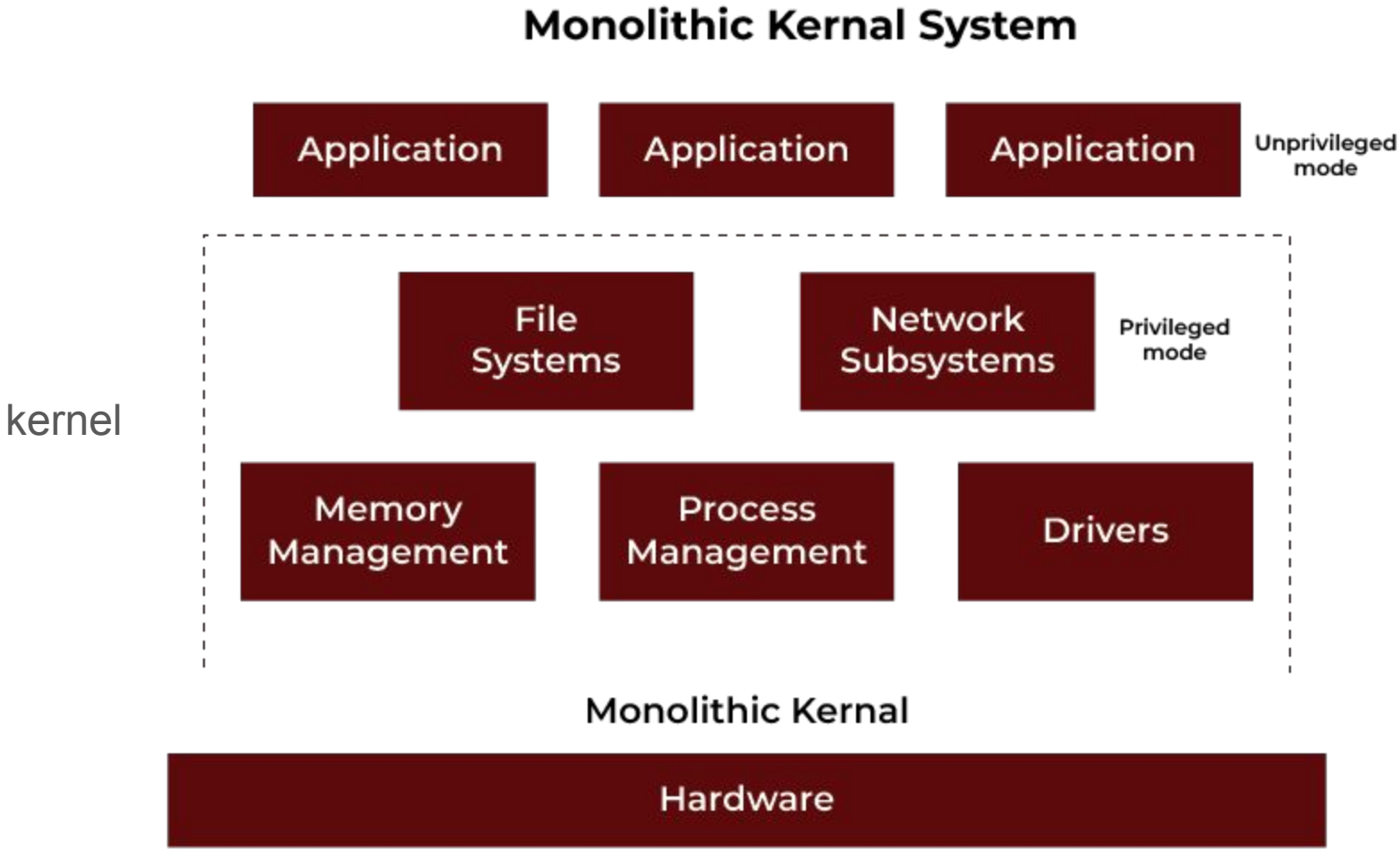
Can be considered hardware
Basic i/o system
Used to start the system .

[Bank example:]
If direct access to h/w Malicious or harmful program can take advantage ; vulnerable

Monolithic

A monolithic structure is an operating system where all core services like process management, memory management, and device drivers are bundled into a single, large program called the kernel.

- Limited structure (disadvantage)
- Too many function packed into single kernel make very difficult in maintenance.
- In order to fix one problem entire kernel needs to access.


Monolithic Kernal System
Application
Application
Application
Unprivileged mode
File Systems
Network Subsystems
Privileged mode
kernel
Memory Management
Process Management
Drivers
Monolithic Kernal
Hardware

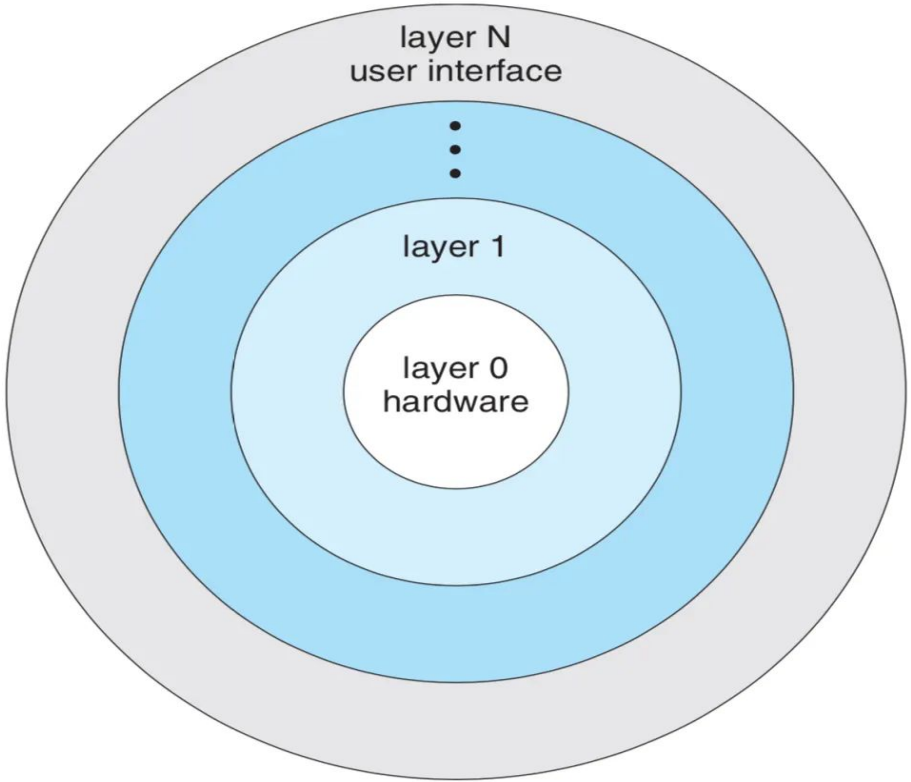
Layered Structure

- A hierarchical model where the OS is divided into distinct levels or layers, starting with the hardware at the bottom **(layer 0)** and ending with the user interface at the top **(layer N)**.

- Each layer can only interact with the layers immediately below it.

- Main advantage easy to debug (only look at that layer and debug, no need to go entire layer)

- Difficulty in design the OS (to decide which will be at the top)



Microkernel

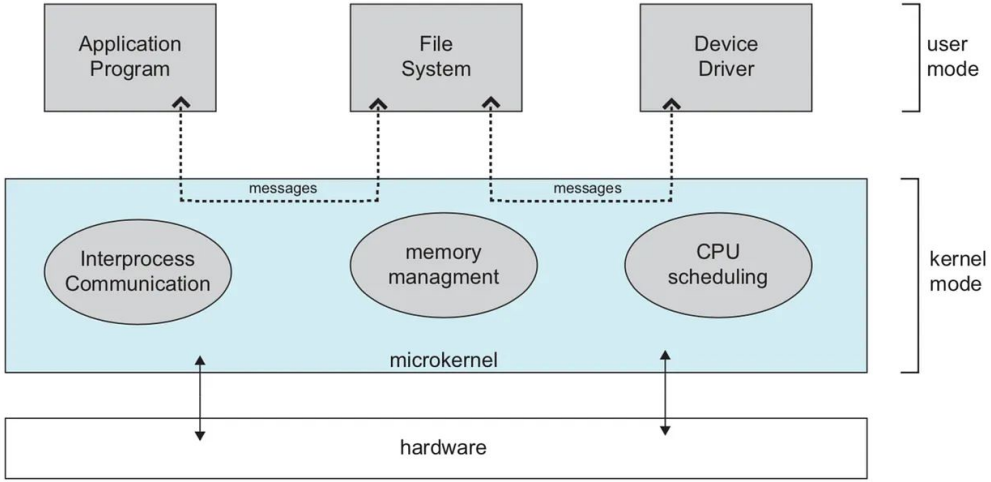
It is a minimalist design that **puts only the most essential functions**, like memory and process management, in the core kernel.

All other services, such as file systems and device drivers, run as separate user-space processes, which enhances security and stability because a failure in one process doesn't crash the whole system.

As most of the functionalities will run in user mode.

Dis adv:

- Performance decrease due to system function.

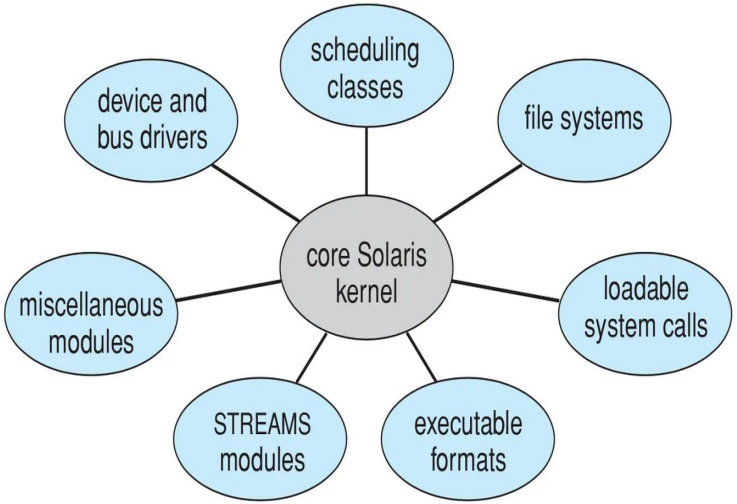


Modular

A modular operating system (OS) is an OS architecture design where the system's functions are divided into independent, **self-contained components** called modules which is loaded dynamically when required.

Flexible than layered structure, here it doesn't have to go.

Advantage as compare to microkernel is that: In microkernel we need to have message passing in order to communicate between modules bcz they are implemented as system or user level program above kernel.



Hybrid

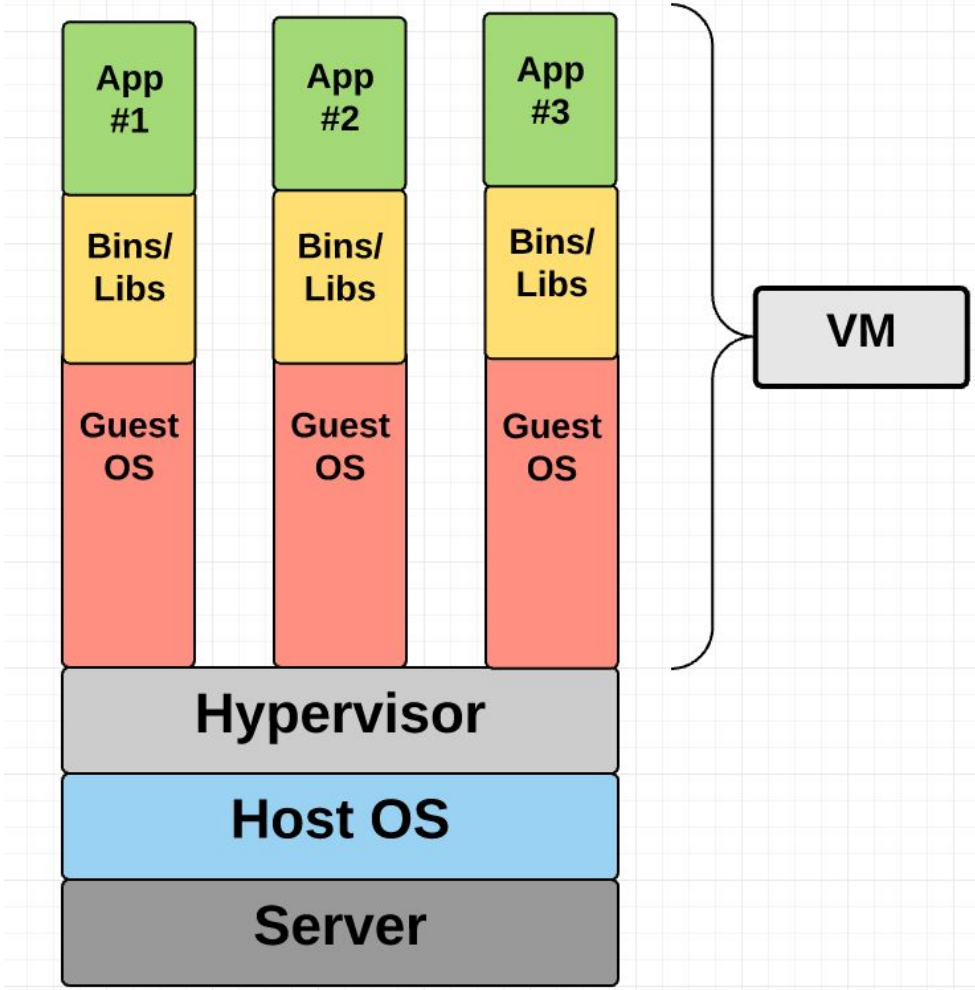
- It is just a combination of both monolithic-kernel structure and microkernel structure.

- It combines properties of both monolithic and microkernel and make a more advanced and helpful approach.

Virtual Machine

- VM is to abstract the hardware of a single computer (the CPU, Memory , disk drives, network interface) into several different execution environments, thereby **creating the illusion that each separate execution** environment is running its own private computer.

- Virtual machines (VMs) allow a business to run an operating system that behaves like a completely separate computer in an app window on a desktop


App #1
App #2
App #3
Bins/ Libs
Bins/ Libs
Bins/ Libs
Guest OS
Guest OS
Guest OS
VM
Hypervisor
Host OS
Server

What are virtual machines used for?

- Run apps requiring different operating systems.
- Test software in a safe, isolated environment.
- Run legacy applications.
- Used heavily in cloud computing.

Container

Light-weight.

Share host OS kernel.

Fast boot, small size.

Ideal for web apps, DevOps, microservices.

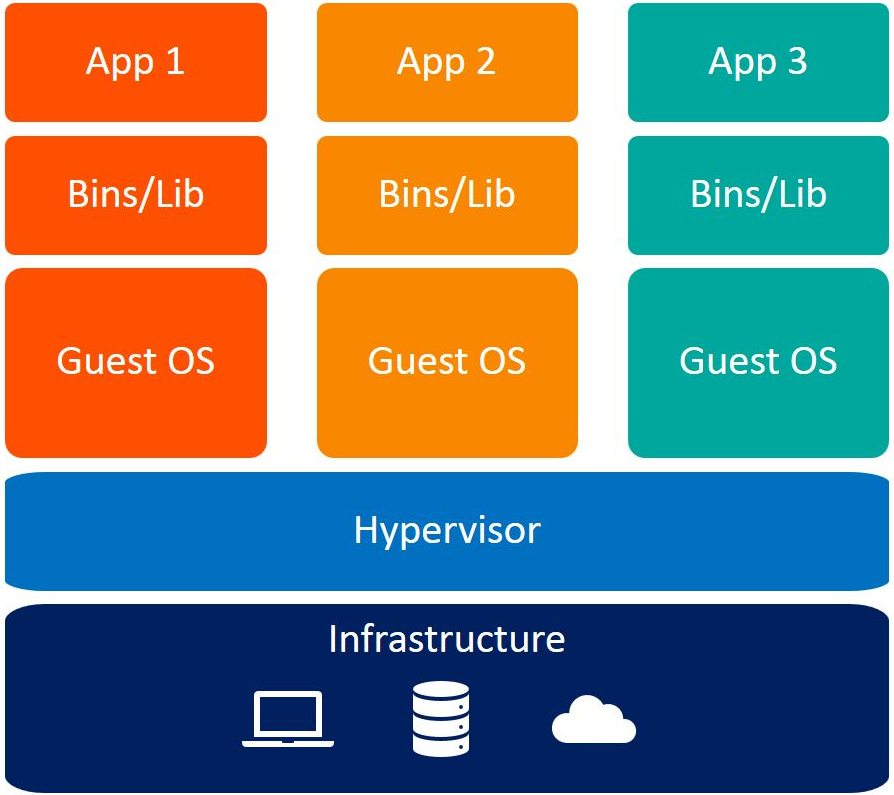
vs.

Virtual machine

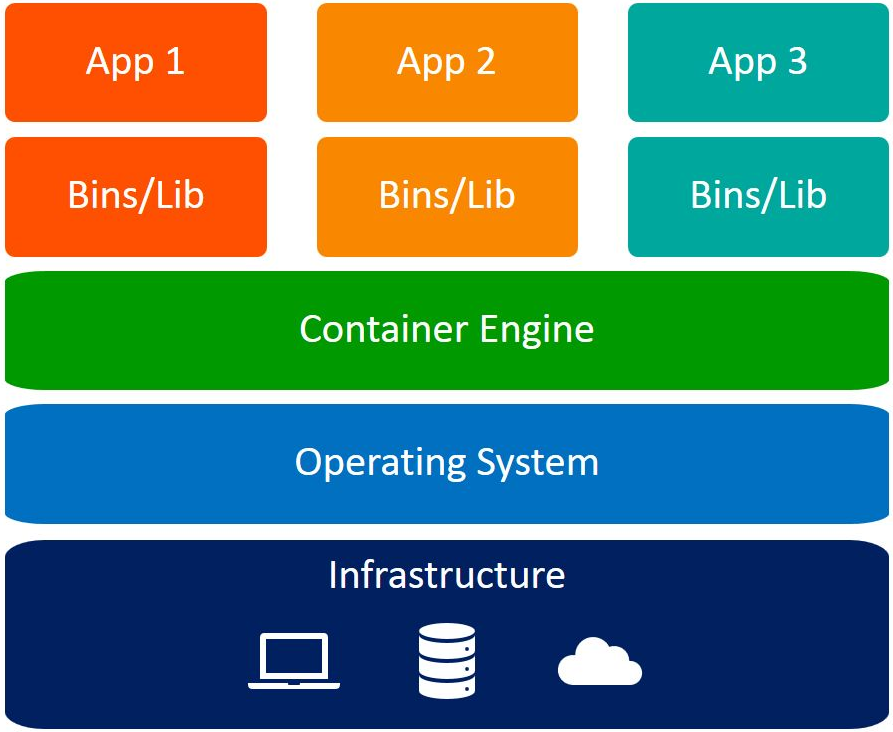
Heavy-weight, include full OS.

Strong isolation.

Good for multiple apps, legacy systems.



Virtual Machines



Containers